Chapter 1

Introduction

Today's software comes with extensive documentation: user guides, reference manuals, and design documents. There are on-line help systems, interactive tutorials, and friendly 'introductions for dummies'. Yet the behaviour of software is often a surprise to users and designers alike. Components interact and interfere, undesirable properties emerge, and systems fail to meet their requirements.

The more spectacular consequences make the headlines: aircraft crash, trains collide, people receive fatal doses of radiation, and emergency telephone services are withdrawn. The less spectacular we face every day: time is wasted, effort is expended to no avail, important projects are scrapped, and our health is damaged by sheer frustration. All of this, and more, because software fails to live up to our expectations.

There are many explanations for this: the requirements upon a piece of software are hard to define, the ways in which a system may be used are hard to anticipate, and there is always a demand for additional functionality. Indeed, the fact that many pieces of software actually work, and work well, is some indication of the skill of those whose job it is to develop them.

1.1 Formal methods

One way to improve the quality of software is to change the way in which software is documented: at the design stage, during development, and after release. Existing methods of documentation offer large amounts of text, pictures, and diagrams, but these are often imprecise and ambiguous. Important information is hidden amongst irrelevant detail, and design flaws are discovered too late, making them expensive or impossible to correct.

There is an alternative. Formal methods, based upon elementary mathematics, can be used to produce precise, unambiguous documentation, in which information is structured and presented at an appropriate level of abstraction. This documentation can be used to support the design process, and as a guide to subsequent development, testing, and maintenance.

It seems likely that the use of formal methods will become standard practice in software engineering. The mathematical basis is different from that of civil or mechanical engineering, but it has the same purpose: to add precision, to aid understanding, and to reason about properties of a design. Whatever the discipline, the use of mathematics can be expensive, but it is our experience that it can actually reduce costs.

Existing applications of formal methods include: the use of probability theory in performance modelling; the use of context-free grammars in compiler design; the use of the relational calculus in database theory. The formal method described in this book has been used in the specification and design of large software systems. It is intended for the description of state and state-based properties, and includes a theory of refinement that allows mathematics to be used at every stage of program development.

1.3 The Z notation

The Z notation is based upon set theory and mathematical logic. The set theory used includes standard set operators, set comprehensions, Cartesian products, and power sets. The mathematical logic is a first-order predicate calculus. Together, they make up a mathematical language that is easy to learn and to apply. However, this language is only one aspect of Z.

Another aspect is the way in which the mathematics can be structured. Mathematical objects and their properties can be collected together in schemas: patterns of declaration and constraint. The schema language can be used to describe the state of a system, and the ways in which that state may change. It can also be used to describe system properties, and to reason about possible refinements of a design.

A characteristic feature of Z is the use of types. Every object in the mathematical language has a unique type, represented as a maximal set in the current specification. As well as providing a useful link to programming practice, this notion of types means that an algorithm can be written to check the type of every object in a specification; several type-checking tools exist to support the practical use of Z.

A third aspect is the use of natural language. We use mathematics to state the problem, to discover solutions, and to prove that the chosen design meets the specification. We use natural language to relate the mathematics to objects in the real world; this job is often partly achieved by the judicious naming of variables, but additional commentary is vital. A well-written specification should be perfectly obvious to the reader.

A fourth aspect is refinement. We may develop a system by constructing a model of a design, using simple mathematical data types to identify the desired behaviour. We may then refine this description by constructing another model which respects the design decisions made, and yet is closer to implementation. Where appropriate, this process of refinement can be continued until executable code is produced.

The Z notation, then, is a mathematical language with a powerful structuring mechanism. In combination with natural language, it can be used to produce formal specifications. We may reason about these specifications using the proof techniques of mathematical logic. We may also refine a specification, yielding another description that is closer to executable code.

Z is not intended for the description of non-functional properties, such as usability, performance, size, and reliability. Neither is it intended for the description of timed or concurrent behaviour. However, there are other formal methods that are well suited for these purposes. We may use these methods in combination with Z to relate state and state-change information to complementary aspects of design.

1.4 The importance of proof

In this book, we place considerable emphasis upon proof. When we introduce the language of mathematical logic, we explain the use of a proof system. When we introduce the language of sets and relations, we explain how formal proofs may be constructed about such objects. When we introduce the language of schemas, we show how to prove that a specification is consistent, and how to prove that one specification refines another. Our intentions are two-fold: first, to show that proof adds quality to software development; second, to show that proof is a feasible part of the industrial use of formal methods.

If we reason about a specification, if we attempt to construct proofs about its properties, then we are more likely to detect problems at an early stage of system development. The process of constructing proofs can help us to understand the requirements upon a system, and can assist us in identifying any hidden assumptions. Proof at the specification stage can make a significant contribution to the quality of software.

At the design stage, a proof can show us not only that a design is correct, but also why it is correct. The additional insight that this affords can be invaluable: as requirements evolve and the design is modified, the consequences are easier to investigate. At the implementation stage, a proof can help us to ensure that a piece of code behaves according to the specification. Again, a significant contribution to quality can be made.

The construction of proofs is an essential part of writing a specification, just as proof-reading is an essential part of writing a book. A specification without proofs is untested: it may be inconsistent; it may describe properties that were not intended, or omit those that were; it may make inappropriate assumptions. The practice of proof makes for better specifications.

It seems to be part of software engineering folklore that proof is impossible on an industrial scale; however, our experience has been different. We have been involved in many large-scale applications of formal methods; some involved proof, others did not. We have seen that techniques involving proof are successful where formal methods are used with a light touch, and where proofs are conducted at an appropriate level of formality.

In many situations, a rigorous argument, or a semi-formal justification, will be sufficient to bring about the desired improvement in quality. In other, more critical situations, it may be necessary to increase the level of formality until the correctness of the design is beyond doubt. In some situations, a completely formal proof may be required. The trick of using formal methods effectively is to know when proofs are worth doing and when they are not.

Chapter 2

Propositional Logic

In this chapter we introduce a logical language based upon traditional propositional calculus. This language is part of the logical language of Z; other parts appear in subsequent chapters. Our presentation is based upon inference and proof: each component of the language is presented alongside rules that explain when it may be introduced or eliminated.

Collected together, these rules form a system of natural deduction: they state what may be deduced from a proposition, and under what conditions that proposition may be concluded. This provides a framework for reasoning about statements in our language, proving properties and establishing results.

2.1 Propositional logic

Propositional logic deals with the statement of alleged facts which must be either true or false, but not both.

Example 2.1 The following statements are propositions:

- A tomato is a fruit.
- An orange is a fruit.
- Oranges are not the only fruit.

□

In our logical language, propositions may be connected in various ways. The following table describes five propositional connectives, arranged in descend-

ing order of operator precedence:

$\neg$	negation	not
$\wedge$	conjunction	and
$\vee$	disjunction	or
$\Rightarrow$	implication	implies
$\Leftrightarrow$	equivalence	if and only if

The table gives the connective's symbol, its name, and how it is pronounced. Using the notion of precedence, we can see that the proposition

$$\neg p \wedge q \vee r \Leftrightarrow q \Rightarrow p \wedge r$$

is equivalent to the parenthesised version

$$(((\neg p) \wedge q) \vee r) \Leftrightarrow (q \Rightarrow (p \wedge r))$$

Using these connectives, we can build up compound propositions.

Example 2.2

- $\neg$(jaffa cakes are biscuits)
- your cat is rich $\wedge$ your dog is good looking
- the economic recovery has started $\vee$ the minister is lying
- Jim is thirty-something $\Rightarrow$ Jim is under forty
- Jim is thirty-something $\Leftrightarrow$ Jim is under forty

□

The truth of a compound proposition is uniquely determined by the truth of its constituent parts.

2.2 Conjunction

In the semantics of Z, a formal meaning is given to propositions; we don't propose to reproduce this here, but rather to give an informal understanding. For example, the conjunction $p \wedge q$ is true exactly when p is true and q is true. In case the reader thinks that this might be a sleight of hand—defining $\wedge$ in

terms of 'and'—we can explain it by using a truth table:

p	q	$p \wedge q$
t	t	t
t	f	f
f	t	f
f	f	f

In the first column we describe all the situations that we are interested in: all the combinations of the possible truth values of p and q (abbreviating truth and falsity to t and f). In the second we have written down the truth value of $p \wedge q$ in each of these situations. Thus, $p \wedge q$ is true just in the case that p is true and q is true.

Now, suppose that we wanted to prove that $p \wedge q$ is true: the truth table tells us when that is so. If we follow the row that has the entry t for $p \wedge q$, we see that we must prove that both p and q are true (have the entry t). Thus, to prove $p \wedge q$, we must prove both p and also q. Now, suppose that we know that $p \wedge q$ is true, then we certainly know that p must be true; we also know that q must be true. We can summarise these observations with the following rules of inference:

$$\frac{p \quad q}{p \wedge q}\ [\wedge\text{–intro}] \qquad \frac{p \wedge q}{p}\ [\wedge\text{–elim1}] \qquad \frac{p \wedge q}{q}\ [\wedge\text{–elim2}]$$

These inference rules form part of the natural deduction system that we use to conduct our proofs.

Each inference rule is written in the following form:

$$\frac{\mathit{premiss}_1 \quad \ldots \quad \mathit{premiss}_n}{\mathit{conclusion}}\ [\text{name}] \qquad \mathit{side\ condition}$$

The list of premisses is sometimes empty; the role of the side condition will become clear later. The meaning of such a rule is that the truth of the conclusion follows from the truth of the premisses: whenever the premisses are true, then so is the conclusion.

The rules come in two flavours. For an operator op, the op-elimination rule describes what may be deduced from $p\ op\ q$; and the op-introduction rule describes under what conditions $p\ op\ q$ can be concluded. Using these rules to introduce and eliminate different operators, we can start from a set of propositions, or hypotheses, and derive another proposition. If the set of hypotheses is empty, then we call the derived proposition a theorem.

We now have two ways of proving things about a proposition. Consider the conjecture that conjunction is commutative. This means that it doesn't matter which way round we present the operands to the conjunction operator: $p \wedge q$ is the same as $q \wedge p$. We prove this first by constructing a truth table, and second by using the rules that we have for introducing and eliminating the conjunction operator.

Example 2.3 First, consider the two truth tables:

p	q	$p \wedge q$
t	t	t
t	f	f
f	t	f
f	f	f

q	p	$q \wedge p$
t	t	t
f	t	f
t	f	f
f	f	f

We can abbreviate the two tables by displaying them in one:

p	q	$p \wedge q$	$q \wedge p$
t	t	t	t
t	f	f	f
f	t	f	f
f	f	f	f

Notice that the columns for $p \wedge q$ and $q \wedge p$ are identical: in every situation they take the same truth value; thus, we can conclude that they are the same proposition, and so conjunction is commutative. □

Now, we shall prove that conjunction is commutative by using a natural deduction argument.

Example 2.4 We want to prove that $p \wedge q$ is equivalent to $q \wedge p$, and this may be deduced from the rule:

$$\frac{p \wedge q}{q \wedge p}$$

We prove this derived rule by exhibiting a proof tree, built from rules fitted together jigsaw-wise. The leaves (at the top of the tree) are instances of the premisses, and the root (at the bottom of the tree) is the conclusion.

$$\dfrac{\dfrac{p \wedge q}{q}\,[\wedge\text{–elim2}] \qquad \dfrac{p \wedge q}{p}\,[\wedge\text{–elim1}]}{q \wedge p}\,[\wedge\text{–intro}]$$

In this tree, there are three rules which have been used. Where the conclusion of one rule forms the premiss of the next, they match exactly. The tree matches the rule that we were trying to prove, since, if we take all the leaves and remove duplicates, we end up with the premiss of the rule; the root of the tree corresponds to the conclusion. □

One more piece of terminology and its notation. Some premisses are introduced during a proof: they are called assumptions. An assumption must be discharged during a proof, and there are certain rules (discharge rules) which do this. The assumption p is denoted by $\lceil p \rceil^{[]}$. In the next section we see examples of assumptions and discharge rules.

2.3 Disjunction

The disjunction $p \vee q$ is true if and only if p is true or q is true:

p	q	$p \vee q$
t	t	t
t	f	t
f	t	t
f	f	f

This is inclusive or: the disjunction is true in any situation in which one of the disjuncts is true, including the situation in which both are true. The disjunction $p \vee q$ is true if and only if either p is true or q is true. Our three rules are:

$$\frac{p}{p \vee q}\ [\vee\text{–intro1}] \qquad \frac{q}{p \vee q}\ [\vee\text{–intro2}] \qquad \frac{p \vee q \qquad \begin{array}{c}\lceil p \rceil^{[i]} \\ r\end{array} \qquad \begin{array}{c}\lceil q \rceil^{[i]} \\ r\end{array}}{r}\ [\vee\text{–elim}^{[i]}]$$

Both introduction rules hold because, if p is true, then $p \vee q$ is true; similarly for q. The elimination rule follows by supposing that $p \vee q$ is true; then, either p or q holds (we don't know which, and it might be both). Whatever follows from this (say r) must follow in both cases. Thus, the elimination rule is an example of case analysis. In the premiss

$$\begin{array}{c}\lceil p \rceil^{[i]} \\ r\end{array}$$

the notation $\lceil p \rceil^{[i]}$ indicates that p is an assumption which may be made in order to prove r. The superscript i indicates that this assumption is justified

by a step labelled i in the proof tree. It is discharged from the proof of r when the rule is applied: given a proof of $p \vee q$, a proof of r from the assumption p, and a proof of r from the assumption q, the rule concludes r.

Example 2.5 Disjunction is commutative:

p	q	$p \vee q$	$q \vee p$
t	t	t	t
t	f	t	t
f	t	t	t
f	f	f	f

The entries for $p \vee q$ and $q \vee p$ are identical. □

Example 2.6 Disjunction is commutative:

$$\frac{p \vee q}{q \vee p}$$

$$\dfrac{p \vee q \qquad \dfrac{\lceil p \rceil^{[1]}}{q \vee p}\,[\vee\text{–intro2}] \qquad \dfrac{\lceil q \rceil^{[1]}}{q \vee p}\,[\vee\text{–intro1}]}{q \vee p}\,[\vee\text{–elim}^{[1]}]$$

□

2.4 Implication

The implication $p \Rightarrow q$ may be viewed as expressing an ordering between the antecedent p and the consequent q: it states that the antecedent is stronger than (or equal to) the consequent. False is stronger than true; true is weaker than false; anything is as strong as itself. This gives the truth table

p	q	$p \Rightarrow q$
t	t	t
t	f	f
f	t	t
f	f	t

Thus, the implication is true unless the antecedent is true and the consequent is false.

The implication $p \Rightarrow q$ is true if and only if we can prove q by assuming p. Thus, in order to prove that $p \Rightarrow q$, we may assume that p is true and then prove that q is true also. If we know that $p \Rightarrow q$, then we can conclude that q is true, providing that we show that the assumption p holds. This gives us the two rules for implication:

$$\frac{\begin{array}{c}[p]^{[i]} \\ q\end{array}}{p \Rightarrow q}\ [\Rightarrow\text{–intro}^{[i]}] \qquad \frac{p \Rightarrow q \quad p}{q}\ [\Rightarrow\text{–elim}]$$

Example 2.7 We can replace a conjunction of antecedents in an implication by separate antecedents:

$$(p \wedge q \Rightarrow r) \Rightarrow (p \Rightarrow (q \Rightarrow r))$$

We may prove this by constructing a truth table:

p	q	r	$(p \wedge q \Rightarrow r) \Rightarrow (p \Rightarrow (q \Rightarrow r))$
t	t	t	$t\ t\ t\ t\ t$
t	t	f	$t\ f\ t\ f\ f$
t	f	t	$f\ t\ t\ t\ t$
t	f	f	$f\ t\ t\ t\ t$
f	t	t	$f\ t\ t\ t\ t$
f	t	f	$f\ t\ t\ t\ f$
f	f	t	$f\ t\ t\ t\ t$
f	f	f	$f\ t\ t\ t\ t$

Every entry in the column underneath the major connective in the proposition is a t: thus the proposition is true in every situation. □

Example 2.8 We can replace a conjunction of antecedents in an implication by separate antecedents:

$$(p \wedge q \Rightarrow r) \Rightarrow (p \Rightarrow (q \Rightarrow r))$$

To see how this might be established, consider the incomplete proof tree:

$$\frac{\vdots}{(p \wedge q \Rightarrow r) \Rightarrow (p \Rightarrow (q \Rightarrow r))}$$

The major connective is an implication; we could consider how it got there, and try to introduce it:

$$\frac{\begin{array}{c}\lceil p \wedge q \Rightarrow r \rceil^{[1]} \\ \vdots \\ (p \Rightarrow (q \Rightarrow r))\end{array}}{(p \wedge q \Rightarrow r) \Rightarrow (p \Rightarrow (q \Rightarrow r))}\ [\Rightarrow\text{–intro}^{[1]}]$$

This leaves us with a new goal, $p \Rightarrow (q \Rightarrow r)$, in which the major connective is again an implication. We follow the same procedure as before; we consider how this operator may be introduced:

$$\frac{\begin{array}{c}\lceil p \wedge q \Rightarrow r \rceil^{[1]} \\ \begin{array}{c}\lceil p \rceil^{[2]} \\ \vdots \\ q \Rightarrow r\end{array}\ [\Rightarrow\text{–intro}^{[2]}] \\ (p \Rightarrow (q \Rightarrow r))\end{array}}{(p \wedge q \Rightarrow r) \Rightarrow (p \Rightarrow (q \Rightarrow r))}\ [\Rightarrow\text{–intro}^{[1]}]$$

Again the goal is an implication. Using the introduction rule a third time leaves us in the following situation:

$$\frac{\begin{array}{c}\lceil p \wedge q \Rightarrow r \rceil^{[1]} \\ \begin{array}{c}\lceil p \rceil^{[2]} \\ \begin{array}{c}\lceil q \rceil^{[3]} \\ \vdots \\ r\end{array}\ [\Rightarrow\text{–intro}^{[3]}] \\ q \Rightarrow r\end{array}\ [\Rightarrow\text{–intro}^{[2]}] \\ (p \Rightarrow (q \Rightarrow r))\end{array}}{(p \wedge q \Rightarrow r) \Rightarrow (p \Rightarrow (q \Rightarrow r))}\ [\Rightarrow\text{–intro}^{[1]}]$$

At this stage, the structure of the goal suggests nothing: there is no structure. Now is the time to start working forwards from the assumptions: one of them has an implication, so we should try to eliminate that:

$$\dfrac{\dfrac{\dfrac{

reminiscent of the rules for conjunction).

$$\frac{p \Rightarrow q \quad q \Rightarrow p}{p \Leftrightarrow q}\,[\Leftrightarrow\text{–intro}] \qquad \frac{p \Leftrightarrow q}{p \Rightarrow q}\,[\Leftrightarrow\text{–elim1}] \qquad \frac{p \Leftrightarrow q}{q \Rightarrow p}\,[\Leftrightarrow\text{–elim2}]$$

Example 2.9 If p is stronger than q, then $p \wedge q$ and p have the same strength:

$$\frac{p \Rightarrow q}{p \wedge q \Leftrightarrow p}$$

To show that this is a derived rule of our system, consider the goal

$$\frac{\vdots}{p \wedge q \Leftrightarrow p}$$

The major connective is the equivalence, so let's try to introduce it:

$$\frac{\dfrac{\vdots}{p \wedge q \Rightarrow p} \quad \dfrac{\vdots}{p \Rightarrow p \wedge q}\,[\Leftrightarrow\text{–intro}]}{p \wedge q \Leftrightarrow p}$$

In the left-hand subtree, the major connective is now an implication, so let's try to introduce that:

$$\frac{\dfrac{\dfrac{\lceil p \wedge q \rceil^{[1]}}{\vdots}}{p}\,[\Rightarrow\text{–intro}^{[1]}]}{p \wedge q \Rightarrow p} \quad \frac{\dfrac{\vdots}{p \Rightarrow p \wedge q}\,[\Rightarrow\text{–intro}]}{[\Leftrightarrow\text{–intro}]}}{p \wedge q \Leftrightarrow p}$$

The left-hand subtree may now be completed by conjunction elimination on the assumption. Turning now to the right-hand subtree, we should immediately introduce the implication:

$$\frac{\dfrac{\dfrac{\lceil p \wedge q \rceil^{[1]}}{p}\,[\wedge\text{–elim1}]}{p \wedge q \Rightarrow p}\,[\Rightarrow\text{–intro}^{[1]}]}{p \wedge q \Leftrightarrow p} \quad \frac{\dfrac{\dfrac{\lceil p \rceil^{[2]}}{\vdots}}{p \wedge q}}{p \Rightarrow p \wedge q}\,[\Rightarrow\text{–intro}^{[2]}]}{[\Leftrightarrow\text{–intro}]}$$

Now, the major connective is a conjunction, so we introduce it:

$$\dfrac{\dfrac{\dfrac{[p \wedge q]^{[1]}}{p}\,[\wedge\text{-elim1}]}{p \wedge q \Rightarrow p}\,[\Rightarrow\text{-intro}^{[1]}] \qquad \dfrac{\dfrac{\overset{[p]^{[2]}}{\vdots}}{p} \quad \dfrac{\overset{[p]^{[2]}}{\vdots}}{q}\,[\wedge\text{-intro}]}{p \wedge q}\,[\Rightarrow\text{-intro}^{[2]}]}{p \wedge q \Leftrightarrow p}\,[\Leftrightarrow\text{-intro}]$$

The left-most unfinished subtree can be closed easily, since we have to prove p from the assumption p: that is immediate. The right-most one cannot be pushed further backwards, since there is no structure to exploit; instead, we work from our premiss:

$$\dfrac{\dfrac{\dfrac{[p \wedge q]^{[1]}}{p}\,[\wedge\text{-elim1}]}{p \wedge q \Rightarrow p}\,[\Rightarrow\text{-intro}^{[1]}] \qquad \dfrac{\dfrac{p \Rightarrow q \quad \dfrac{\overset{[p]^{[2]}}{\vdots}}{p}\,[\Rightarrow\text{-elim}]}{q}\,[\wedge\text{-intro}]}{p \wedge q}\,[\Rightarrow\text{-intro}^{[2]}]}{p \wedge q \Leftrightarrow p}\,[\Leftrightarrow\text{-intro}]$$

Again, the closing of this subtree is trivial, thus completing the proof:

$$\dfrac{\dfrac{\dfrac{[p \wedge q]^{[1]}}{p}\,[\wedge\text{-elim1}]}{p \wedge q \Rightarrow p}\,[\Rightarrow\text{-intro}^{[1]}] \qquad \dfrac{\dfrac{p \Rightarrow q \quad [p]^{[2]}}{q}\,[\Rightarrow\text{-elim}] \quad [p]^{[2]}}{p \wedge q}\,[\wedge\text{-intro}]}{p \wedge q \Leftrightarrow p}\,[\Rightarrow\text{-intro}^{[2]}]\,[\Leftrightarrow\text{-intro}]$$

□

A derived rule may be used in the same way as any other inference rule; the above example gives us

$$\dfrac{p \Rightarrow q}{p \wedge q \Leftrightarrow p}\,[\text{subsume}]$$

This is just one of several similar inference rules involving conjunction, disjunction, and implication.

2.6 Negation

The negation $\neg p$ is true if and only if p is false. The truth table is simple:

p	$\neg p$
t	f
f	t

Our rules for negation make use of a special proposition called *false*, which stands for a contradiction: it is false in every situation. If $\neg p$ is true, then p is false; and if p is true, then $\neg p$ is false. Notice that it is not possible for $\neg p$ and p both to be true. This gives us three rules:

$$\frac{\begin{array}{c}[p]^{[i]}\\ \mathit{false}\end{array}}{\neg p}\ [\neg\text{–intro}^{[i]}] \qquad \frac{p \quad \neg p}{\mathit{false}}\ [\neg\text{–elim}] \qquad \frac{\begin{array}{c}[\neg p]^{[j]}\\ \mathit{false}\end{array}}{p}\ [\mathit{false}\text{–elim}^{[j]}]$$

Our system requires three rules to deal with negation. At first sight, it might seem that the two rules that we have called $\neg$–*intro* and *false*–*elim* would be sufficient, but they would give us no way of concluding *false*.

Example 2.10 One of de Morgan's Laws states that the negation of a disjunction is the conjunction of negations:

$$\frac{\neg(p \vee q)}{\neg p \wedge \neg q}\ [\text{de Morgan1}]$$

We start by considering the goal:

$$\frac{\vdots}{\neg p \wedge \neg q}$$

Clearly, we should break up the conjunction:

$$\frac{\begin{array}{cc}\vdots & \vdots\\ \neg p & \neg q\end{array}}{\neg p \wedge \neg q}\ [\wedge\text{–intro}]$$

Let's focus on the left subtree. In order to prove the negation $\neg p$, we should assume p and then force a contradiction:

$$\dfrac{\dfrac{\dfrac{[p]^{[1]}}{\vdots}}{\dfrac{\mathit{false}}{\neg p}\,[\neg\text{-intro}^{[1]}]} \quad \dfrac{\vdots}{\neg q}}{\neg p \wedge \neg q}\,[\wedge\text{-intro}]\;[\neg\text{-elim}]$$

Now, what should the contradiction be? We have the premiss $\neg(p \vee q)$, we could try to contradict that:

$$\dfrac{\dfrac{\dfrac{\dfrac{[p]^{[1]}}{\vdots}}{p \vee q} \quad \neg(p \vee q)}{\dfrac{\mathit{false}}{\neg p}\,[\neg\text{-intro}^{[1]}]}\,[\neg\text{-elim}] \quad \dfrac{\vdots}{\neg q}}{\neg p \wedge \neg q}\,[\wedge\text{-intro}]$$

We can close this subtree by noting that we can prove $p \vee q$ from p:

$$\dfrac{\dfrac{\dfrac{[p]^{[1]}}{p \vee q}\,[\vee\text{-intro1}] \quad \neg(p \vee q)}{\dfrac{\mathit{false}}{\neg p}\,[\neg\text{-intro}^{[1]}]}\,[\neg\text{-elim}] \quad \dfrac{\vdots}{\neg q}}{\neg p \wedge \neg q}\,[\wedge\text{-intro}]$$

The rest of the proof follows by symmetry:

$$\dfrac{\dfrac{\dfrac{\dfrac{[p]^{[1]}}{p \vee q}\,[\vee\text{-intro1}] \quad \neg(p \vee q)}{\dfrac{\mathit{false}}{\neg p}\,[\neg\text{-intro}^{[1]}]}\,[\neg\text{-elim}] \quad \dfrac{\dfrac{[q]^{[2]}}{p \vee q}\,[\vee\text{-intro2}] \quad \neg(p \vee q)}{\dfrac{\mathit{false}}{\neg q}\,[\neg\text{-intro}^{[2]}]}\,[\neg\text{-elim}]}{\neg p \wedge \neg q}\,[\wedge\text{-intro}]$$

□

Before a natural deduction system becomes really useful, it is necessary to prove results about negation. In the next theorem we use the variant of de Morgan's law that we have just proved.

Example 2.11 The Law of the Excluded Middle states that either a proposition is true or it is false. That is,

$$p \vee \neg p$$

The major connective is a disjunction, so let's try to introduce it. Which disjunct should we throw away? Let's try $\neg p$:

$$\dfrac{\dfrac{\vdots}{p}}{p \vee \neg p}\ [\vee\text{–intro1}]$$

Now there is no structure to analyse, so we are lost, since there are no assumptions or premisses either. We must admit defeat and backtrack to the last choice that we made. Perhaps we should have thrown away p instead:

$$\dfrac{\dfrac{\vdots}{\neg p}}{p \vee \neg p}\ [\vee\text{–intro2}]$$

Now we are in the same position: we still cannot complete the proof. Again, we must backtrack to the last decision point. This time, we go right back to the start of the proof. We have tried both varieties of disjunction introduction, and there are neither assumptions nor premisses to work from. What now? One possible way forward is to try to contradict our goal: assume $\neg(p \vee \neg p)$ and force a contradiction:

$$\dfrac{\dfrac{\begin{array}{c}[\neg(p \vee \neg p)]^{[1]} \\ \vdots \\ \mathit{false}\end{array}}{p \vee \neg p}\ [\mathit{false}\text{–elim}^{[1]}]}$$

Our contradiction follows by Example 2.10:

$$\dfrac{\dfrac{\dfrac{[\neg(p \vee \neg p)]^{[1]}}{\neg p \wedge \neg\neg p}\ [\text{de Morgan1}] \\ \vdots \\ \mathit{false}}{p \vee \neg p}\ [\neg\text{–elim}]\ [\mathit{false}\text{–elim}^{[1]}]}$$

The proof tree isn't quite closed, because of the technicality that we need two separate propositions: $\neg p$ and $\neg\neg p$. We can get each from the conjunction, so we need to duplicate our work from the assumption, and use both varieties of conjunction elimination:

$$\dfrac{\dfrac{\dfrac{[\neg(p \vee \neg p)]^{[1]}}{\neg p \wedge \neg\neg p}\,[\text{de Morgan1}]}{\neg p}\,[\wedge\text{−elim1}] \qquad \dfrac{\dfrac{[\neg(p \vee \neg p)]^{[1]}}{\neg p \wedge \neg\neg p}\,[\text{de Morgan1}]}{\neg\neg p}\,[\wedge\text{−elim2}]}{\dfrac{\mathit{false}}{p \vee \neg p}\,[\mathit{false}\text{−elim}^{[1]}]}\,[\neg\text{−elim}]$$

Now we have finished, and the proof tree is complete. □

The last example shows an important part of the proof process: the exploration of possibilities. When we look at a proof, we see only a completed chain of reasoning; we do not see the other attempts that may have been made. Furthermore, rules like negation elimination give us a problem in a backwards proof, since p appears in the premiss, but not in the conclusion. Thus, when we match the consequent to our current goal, we still have to find an instantiation of p.

2.7 Tautologies and contradictions

Propositions which evaluate to t in every combination of their propositional variables are known as tautologies: they are always true. If, on the other hand, they evaluate to f in every combination, then they are known as contradictions. Of course, the negation of a contradiction is a tautology, and vice versa.

Example 2.12 The following propositions are tautologies:

$$p \vee \neg p$$

$$p \Rightarrow p$$

$$p \Rightarrow (q \Rightarrow p)$$

while the following are contradictions:

$$p \wedge \neg p$$

$$p \Leftrightarrow \neg p$$

$$\neg(p \Rightarrow (q \Rightarrow p))$$

□

To prove that a proposition is a tautology, we have only to produce a truth table and check that the major connective takes the value t for each combination of propositional variables.

Example 2.13 We prove that $\neg p \vee q \Leftrightarrow p \Rightarrow q$ is a tautology by exhibiting the following table:

p	q	$\neg p \vee q \Leftrightarrow p \Rightarrow q$			
t	t	f	t	t	t
t	f	f	f	t	f
f	t	t	t	t	t
f	f	t	t	t	t

□

Tautologies involving equivalences are particularly useful in proofs; they can be used to rewrite goals and assumptions to facilitate the completion of an argument. For any pair of propositions a and b, the tautology $a \Leftrightarrow b$ corresponds to a pair of inference rules:

$$\frac{b}{a}\,[a \Leftrightarrow b] \qquad \frac{a}{b}\,[a \Leftrightarrow b]$$

If either of these propositions appears in a proof, then we may replace it with the other:

$$\dfrac{\begin{array}{c}\vdots\\ a\end{array}}{\begin{array}{c}b\\ \vdots\end{array}}\,[a \Leftrightarrow b] \qquad \begin{array}{c}\vdots\\ \vdots\end{array}$$

A logical equivalence may be used to justify rewriting even when the proposition involved is only part of the goal or assumption:

$$\dfrac{\begin{array}{c}\vdots\\ (\neg\, p \vee q) \Rightarrow r\end{array}}{\begin{array}{c}(p \Rightarrow q) \Rightarrow r\\ \vdots\end{array}}\,[(\neg a \vee b) \Leftrightarrow (a \Rightarrow b)] \qquad \begin{array}{c}\vdots\\ \vdots\end{array}$$

Tautologies involving implications also correspond to inference rules: if $a \Rightarrow b$ is a tautology, then

$$\frac{a}{b}\ [a \Rightarrow b]$$

may be used as a derived rule. An implication alone is not enough to justify rewriting part of a goal. To see why not, consider the following proposition:

$$(p \wedge q) \Rightarrow (r \vee s)$$

The proposition $(a \wedge b) \Rightarrow a$ is a tautology, but the proof step

$$\frac{(p \wedge q) \Rightarrow (r \vee s)}{p \Rightarrow (r \vee s)}\ [(a \wedge b) \Rightarrow a]$$

is invalid. The statement $p \Rightarrow (r \vee s)$ does not follow from $(p \wedge q) \Rightarrow r \vee s$: it is possible for the former to be false when the latter is true.

Example 2.14 The following tautology

$$\neg(p \wedge q) \Rightarrow \neg p \vee \neg q$$

corresponds to another of de Morgan's laws:

$$\frac{\neg(p \wedge q)}{\neg p \vee \neg q}\ [\text{de Morgan2}]$$

□

A proposition which is neither a tautology nor a contradiction is said to be a contingency.

Chapter 3

Predicate Logic

In this chapter we introduce another part of our logical language. The language of propositions introduced in the previous chapter allows us to make statements about specific objects, but it does not allow us to make statements such as 'Every cloud has a silver lining'. These are known as universal statements, since they describe properties that must be satisfied by every object in some universe of discourse.

Example 3.1 The following are examples of universal statements:

- Each student must hand in course work.
- Nobody knows the trouble I seen.
- Jim doesn't know anybody who can sign his bail application.

□

Sometimes we wish to state that at least one thing has a particular property, without necessarily knowing which thing it is. This leads to an existential statement.

Example 3.2 The following are examples of existential statements:

- I heard it from one of your friends.
- A mad dog has bitten Andy.
- Some people prefer logic.

□

To formalise such statements, we require a language that reveals the internal structure of our propositional statements, a language that allows us to take them apart and apply them to objects without proper names. The language we require is the language of predicate calculus.

3.1 Predicate calculus

A predicate is a statement with a place for an object. There may be many such places within a single predicate; this is often the case when the objects concerned are mathematical. When these places are filled, our predicates become statements about the objects that fill them. We could say that a predicate is a proposition with a gap for an object of some kind.

For example, the statement '$_ > 5$' is a predicate. As it stands, it is not a proposition; we cannot say whether it is true or false until we have filled the empty place. We could turn it into a proposition by putting 0 in this place; the result would be '$0 > 5$', a proposition that happens to be false. This is not the only way to fill a gap, however. We could also choose to put an object variable in the empty place above.

The predicate '$x > 5$' is still not a proposition; we cannot say whether it is true or false without knowing what x is. The use of object variables is a powerful technique, and holds the key to expressing the universal and existential properties described above. We can make a proposition out of '$x > 5$' by adding a quantifier to the front of the expression. For example, we could state that 'there is an x, which is a natural number, such that $x > 5$'. Here, the quantifier is 'there is an…', and we have quantified the predicate '$x > 5$' to produce a true proposition.

In mathematics, the symbol '$\exists$' is used to denote the expression 'there is an …'; in Z, the natural numbers are denoted by the symbol '$\mathbb{N}$'. Thus, we can write down our quantified predicate in Z as:

$$\exists x : \mathbb{N} \bullet x > 5.$$

To see that the quantified predicate is true, consider the number 6: it is a natural number, and it is greater than 5.

Existential quantification may be thought of as a generalised form of disjunction: for example,

$$\exists x : \mathbb{N} \bullet x > 5$$
$$\Leftrightarrow$$
$$0 > 5 \lor 1 > 5 \lor 2 > 5 \lor 3 > 5 \lor \ldots$$

The predicate is true for some natural number; it is true of 0 or it is true of 1 or it is true of 2 or it is true of 3, etcetera.

Example 3.3 The statements in Example 3.2 may be formalised as follows:

- Let *Friends* stand for the set of all your friends, and let x *told* y mean that x has told y.

  $$\exists f : Friends \bullet f \ told \ me$$

- Let *MadDog* stand for the set of all mad dogs, and let x *bit* y mean that x has bitten y.

  $$\exists d : MadDog \bullet d \ bit \ Andy$$

- Let *Person* stand for the set of all people, and let $PL(x)$ mean that x prefers logic.

  $$\exists p : Person \bullet PL(p)$$

□

Another way of quantifying a predicate is to say that it is true for every value. We might take the predicate '$x > 5$' and prepend a universal quantifier to produce the statement 'for every x which is a natural number, it is the case that $x > 5$'. Here, the quantifier is 'for every x ...', and we have quantified the predicate to produce a false proposition.

In mathematics, the notation '$\forall$' is used to denote the universal quantifier. We can write down our new predicate in Z as follows:

$$\forall x : \mathbb{N} \bullet x > 5.$$

Again, this is the same as

$$\forall y : \mathbb{N} \bullet y > 5.$$

This predicate is false because not every natural number x is greater than 5: consider 3.

The universal quantifier may be thought of as a generalised conjunction: for example,

$$\forall x : \mathbb{N} \bullet x > 5$$
$$\Leftrightarrow$$
$$0 > 5 \land 1 > 5 \land 2 > 5 \land 3 > 5 \land \ldots$$

The predicate $_ > 5$ would have to be true of every natural number; of 0 and of 1 and of 2 and of 3, etcetera. It is not true of 0, for example, and thus the whole quantified expression is false.

Example 3.4 The statements in Example 3.1 may be formalised as follows:

- Let *Student* stand for the set of all students, and let $Submit(x)$ mean that x must hand in course work.

  $$\forall s : Student \bullet Submit(s)$$

- Let *Person* be the set of all people, and let $knows_trouble(x)$ mean that x knows the trouble I seen.

  $$\forall p : Person \bullet \neg knows_trouble(p)$$

- Again, let *Person* be the set of all people. Let $x\ Knows\ y$ means that x knows y, and let $x\ CanBail\ y$ mean that x can sign y's application for bail.

  $$\forall p : Person \bullet Jim\ Knows\ p \Rightarrow \neg (p\ CanBail\ Jim)$$

□

3.2 Quantifiers and declarations

In the Z notation, the two kinds of quantified expressions have a similar syntax:

$$\mathcal{Q}\, x : a \mid p \bullet q$$

where

- $\mathcal{Q}$ is the quantifier;
- x is the bound variable;
- a is the range of x;
- p is the constraint; and
- q is the predicate.

The optional constraint p restricts the set of objects under consideration; only those objects in a that satisfy p are to be considered. The constraint takes

on the role of a conjunction or an implication, depending upon the quantifier concerned, as may be seen from the following equivalences:

$$(\exists\, x : a \mid p \bullet q) \;\Leftrightarrow\; (\exists\, x : a \bullet p \wedge q)$$
$$(\forall\, x : a \mid p \bullet q) \;\Leftrightarrow\; (\forall\, x : a \bullet p \Rightarrow q)$$

The existentially quantified predicate

$$\exists\, x : a \mid p \bullet q$$

is pronounced 'there exists an x in a satisfying p, such that q'. The universally quantified predicate

$$\forall\, x : a \mid p \bullet q$$

is pronounced 'for all x in a satisfying p, q holds'.

Each quantifier introduces a 'bound variable', which is analogous to a local variable in a block-structured programming language. In the quantified predicate $\mathcal{Q}\ x : a \mid p \bullet q$ the bound variable x has a scope that is exactly the constraint p and predicate q. The quantifiers bind very loosely, so the scope of a quantified variable extends to the next enclosing bracket.

Example 3.5 In the following expression, the scope of variable x is marked by a brace:

$$(\forall\, x : a \mid \underbrace{p \bullet q \wedge r}_{\text{scope of } x}) \vee s \Rightarrow t$$

□

If a statement contains more than one quantifier, the scopes may overlap. This poses no problems unless the same name is chosen for two variables bound by different quantifiers; in this case, there would be a hole in one of the scopes.

Example 3.6 In the following expression, the scope of the first bound variable has a hole corresponding to the scope of the second:

$$\forall\, y : a \mid \underbrace{p \bullet q \wedge (\ \forall\, y : b \mid \overbrace{r \bullet s \Rightarrow t}^{\text{scope of second } y}\ )}_{\text{scope of first } y} \wedge u \vee v$$

□

As the above example shows, there is scope for confusion whenever two different variables have the same name.

Whenever such confusion can arise, we will choose another name for one of the variables. We can change the name of a bound variable without changing the meaning of the quantified expression, as long as we avoid the names of any other variables that appear.

Example 3.7 Consider the following quantified expression, which states that every natural number x is greater than or equal to zero:

$$\forall num : \mathbb{N} \bullet num \geq 0$$

The choice of '*num*' as the variable name is not important; the following expression has the same meaning:

$$\forall nat : \mathbb{N} \bullet nat \geq 0$$

□

We must take care that the new name chosen for a bound variable has not already been used for a different variable in the same expression.

Example 3.8 Consider the following quantified expression, which states that there is some natural number *max* such that every natural number *num* must be less than or equal to *max*:

$$\exists max : \mathbb{N} \bullet \forall num : \mathbb{N} \bullet num \leq max$$

This statement is false: there is no greatest natural number.

If we were to change the name of the universally-quantified variable from *num* to *max*, then some confusion would result. The following expression states that there is some natural number such that every natural number is less than or equal to itself.

$$\exists max : \mathbb{N} \bullet \forall max : \mathbb{N} \bullet max \leq max$$

This statement is true; the meaning has changed. □

To avoid changing the meaning of a statement, we insist that a fresh variable name is chosen whenever such a change of name occurs. This name should not appear elsewhere in the logical expressions under consideration.

In the last example, we saw two variables quantified at the start of an expression, one immediately after the other. In this case, the quantifiers were different. Had they been the same, we could have quantified both variables at the same time, separating their declarations with a semicolon.

Example 3.9 The quantified predicate

$$\exists\, x : a \bullet \exists\, y : b \bullet p$$

could also be written as

$$\exists\, x : a;\ y : b \bullet p$$

□

There is a circumstance in which this is not possible: when the first quantified variable appears in the range of the second.

Example 3.10 In the expression below, the first bound variable is used as the range of the second:

$$\exists\, a : b \bullet \exists\, c : a \bullet p$$

In this case, it would make no sense to merge the two quantifications. □

If a variable x appears in a predicate p but is not bound by any quantifier, we say that x is free in p. Each occurrence of x that is outside the scope of a declaration of the form '$\forall\, x : a$' or '$\exists\, x : a$' is said to be a free occurrence.

Example 3.11 In the expression below, there is a single free occurrence of variable z:

$$\forall\, x : \mathbb{N} \bullet z \leq x$$

This predicate states that every natural number x is greater than z, whatever z may be. □

If we use the same name for two different variables, then we may find that a variable appears both free and bound in the same expression.

Example 3.12 There are both free and bound occurrences of variable x in the expression below:

$$\underbrace{x}_{\text{free occurrence}} = 3 \land \forall\, x : \mathbb{N} \bullet 0 \leq \underbrace{x}_{\text{bound occurrence}}$$

The occurrence of x adjacent to the quantifier is neither free nor bound; it is a binding occurrence. □

3.3 Substitution

If a predicate p contains a free occurrence of variable x, then p may represent a non-trivial statement about x. The choice of variable x is important: p does not, in general, represent the same statement about any other variable y. If we wish to change the subject of the statement from x to y, we must replace each free occurrence of x in p with an occurrence of y. This process is called substitution.

We write $p[y \,/\, x]$ to denote the predicate that results from substituting y for each free occurrence of x in predicate p; this new operator binds more tightly than any other. The expression y need not be another variable; it can be any expression whose possible values match those of x.

Example 3.13

1. $(x \le y + 2)[0 \,/\, x] \;\Leftrightarrow\; (0 \le y + 2)$
2. $(\exists\, x : \mathbb{N} \bullet x \le y + 2)[0 \,/\, x] \;\Leftrightarrow\; (\exists\, x : \mathbb{N} \bullet x \le y + 2)$
3. $(\exists\, x : \mathbb{N} \bullet x \le y + 2)[5 \,/\, y] \;\Leftrightarrow\; (\exists\, x : \mathbb{N} \bullet x \le 5 + 2)$

□

We write $p[t \,/\, x][u \,/\, y]$ to denote the predicate $p[t \,/\, x]$ with the expression u systematically substituted for free occurrences of the variable y.

Example 3.14

1. $(x \le y + 2)[0 \,/\, x][5 \,/\, y] \;\Leftrightarrow\; (0 \le y + 2)[5 \,/\, y] \;\Leftrightarrow\; (0 \le 5 + 2)$
2. $(x \le y + 2)[y \,/\, x][5 \,/\, y] \;\Leftrightarrow\; (y \le y + 2)[5 \,/\, y] \;\Leftrightarrow\; (5 \le 5 + 2)$

□

We write $p[t, u \,/\, x, y]$ to denote the result of simultaneously substituting t for x and u for y in predicate p. In general, this is different from the multiple substitution $p[t \,/\, x][u \,/\, y]$.

Example 3.15

1. $(x \le y + 2)[y, 5 \,/\, x, y] \;\Leftrightarrow\; (y \le 5 + 2)$
2. $(x \le y + 2)[y \,/\, x][5 \,/\, y] \;\Leftrightarrow\; (y \le y + 2)[5 \,/\, y] \;\Leftrightarrow\; 5 \le 5 + 2$

□

A potential problem with substitution is the unintentional capture of free variables. If y is bound in p, then the substitution $p[y \,/\, x]$ might include new bound instances of y in place of free instances of x. This may change the meaning of p in a way that is not intended.

Example 3.16 Let *Person* denote the set of all people, and let m *LooksLike* n mean that person m looks like person n. The following predicate is a statement about a person o; it states that there is some person who does not look like o:

$$\exists\, p : \mathit{Person} \bullet \neg (p \mathrel{\mathit{LooksLike}} o)$$

We may make the same statement about person m by substituting m for o:

$$\exists\, p : \mathit{Person} \bullet \neg (p \mathrel{\mathit{LooksLike}} m)$$

However, if we substitute p for o, we obtain a different statement entirely:

$$\exists\, p : \mathit{Person} \bullet \neg (p \mathrel{\mathit{LooksLike}} p)$$

The expression substituted for o contains a free occurrence of p, which is then bound by the quantifier. The new predicate states that there is someone who does not look like themself. The substitution has brought an unwanted change of meaning. □

To avoid such confusion, we may rename bound variables prior to substitution, choosing fresh variable names to avoid variable capture.

We can give equivalences to explain the effect of substitution into quantified expressions. In the simplest case, the variable being substituted for has the same name as the one being quantified:

$$(\forall\, x : a \mid p \bullet q)[t \,/\, x] \;\Leftrightarrow\; (\forall\, x : a[t \,/\, x] \mid p \bullet q)$$
$$(\exists\, x : a \mid p \bullet q)[t \,/\, x] \;\Leftrightarrow\; (\exists\, x : a[t \,/\, x] \mid p \bullet q)$$

In this case, the only part of the expression that may change is the range of the quantified variable. In general, this substitution will have no effect; it is poor practice to include a free variable in the declaration of a bound variable of the same name.

If the quantifier is binding some variable other than x, then the substitution will have more of an effect. If y is not free in t, then

$$(\forall\, y : a \mid p \bullet q)[t \,/\, x] \;\Leftrightarrow\; (\forall\, y : a[t \,/\, x] \mid p[t \,/\, x] \bullet q[t \,/\, x])$$
$$(\exists\, y : a \mid p \bullet q)[t \,/\, x] \;\Leftrightarrow\; (\exists\, y : a[t \,/\, x] \mid p[t \,/\, x] \bullet q[t \,/\, x])$$

If y is free in t, then we choose a fresh variable z, different from x and not appearing in t:

$$
\begin{array}{l}
(\forall\, y : a \mid p \bullet q)[t \,/\, x] \\
\quad \Leftrightarrow\ (\forall\, z : a[t \,/\, x] \mid p[z \,/\, y][t \,/\, x] \bullet q[z \,/\, y][t \,/\, x]) \\
(\exists\, y : a \mid p \bullet q)[t \,/\, x] \\
\quad \Leftrightarrow\ (\exists\, z : a[t \,/\, x] \mid p[z \,/\, y][t \,/\, x] \bullet q[z \,/\, y][t \,/\, x])
\end{array}
$$

By using z instead of y for the name of the quantified variable, we have avoided any possibility of unintentional variable capture.

If the major operator in an expression is not a quantifier, then the effect of substitution is easy to explain:

$$
\begin{array}{rcl}
(\neg p)[t \,/\, x] & \Leftrightarrow & \neg p[t \,/\, x] \\
(p \wedge q)[t \,/\, x] & \Leftrightarrow & p[t \,/\, x] \wedge q[t \,/\, x] \\
(p \vee q)[t \,/\, x] & \Leftrightarrow & p[t \,/\, x] \vee q[t \,/\, x] \\
(p \Rightarrow q)[t \,/\, x] & \Leftrightarrow & p[t \,/\, x] \Rightarrow q[t \,/\, x] \\
(p \Leftrightarrow q)[t \,/\, x] & \Leftrightarrow & p[t \,/\, x] \Leftrightarrow q[t \,/\, x]
\end{array}
$$

In every case, substitution distributes through the propositional operators.

3.4 Universal introduction and elimination

In general, the truth-table technique for giving meaning to connectives and reasoning about them is useless for the quantifiers, since the sets that bound variables may range over are simply too large. However, we may build upon the natural deduction system of the previous chapter by adding rules to introduce and eliminate quantifiers.

If we view universal quantification as a generalised conjunction, then we should be able to generalise the rules for conjunction to get the rules for the universal quantifier. Consider first the introduction rule. In order to prove $p \wedge q$, one needs to prove both p and q. In order to prove $\forall\, x : a \bullet p$, one must prove that p is true for each value in a. This doesn't sound terribly hopeful, as it might involve an infinite number of premisses, and therefore an infinite number of proofs.

A better approach might be to prove that p holds for an arbitrary member of a: if we make no assumptions whatsoever about which member of a we choose in order to prove p, then our proof generalises to all members. A simplified

rule for introducing universal quantification is the following:

$$\frac{\lceil x \in a \rceil^{[i]} \quad q}{\forall\, x : a \bullet q}\ [^{[i]}] \qquad \text{provided that } x \text{ is not free in the assumptions of } q$$

where $x \in a$ means that x is a member of set a.

Notice that we are required to check that x is not free in the assumptions of q. This ensures that we are not making any assumptions about which member of a we are choosing. The assumptions of q are those leaves of the proof tree above q that have not been discharged: by implication-introduction, for example.

In the full form of the universal quantifier, there is also a constraint that x must satisfy; we may treat the constraint as an additional assumption:

$$\frac{\lceil x \in a \rceil^{[1]} \quad \lceil p \rceil^{[1]} \quad q}{\forall\, x : a \mid p \bullet q}\ [\forall\text{–intro}^{[1]}] \qquad \text{provided that } x \text{ is not free in the assumptions of } q$$

This rule may be derived from the first $\forall$*–intro* rule:

$$\dfrac{\dfrac{\dfrac{\lceil x \in a \rceil^{[1]} \quad \lceil p \rceil^{[2]} \quad \vdots \quad q}{p \Rightarrow q}\ [\Rightarrow\text{–intro}^{[2]}]}{\forall\, x : a \bullet p \Rightarrow q}\ [\forall\text{–intro}^{[1]}]}{\forall\, x : a \mid p \bullet q}\ [\text{defn}]$$

The constraint part of a universal quantification may be treated as the antecedent of an implication.

From a conjunction, one may conclude either of the conjuncts; by analogy, from a universally quantified predicate, one may conclude that the predicate holds for any value in the range. Suppose that we have the universally quantified predicate $\forall\, x : a \bullet p$, and that the expression t denotes a value in a; then p must be true of t.

$$\frac{t \in a \quad \forall\, x : a \bullet p}{p[t \,/\, x]}$$

We systematically substitute t for x in p.

The full form requires the equivalent of implication elimination, to demonstrate that the expression chosen satisfies the constraint:

$$\frac{t \in a \quad \forall\, x : a \mid p \bullet q \quad p[t\,/\,x]}{q[t\,/\,x]}\ [\forall\text{–elim}]$$

A special case of the last rule takes t as x:

$$\frac{x \in a \quad \forall\, x : a \mid p \bullet q \quad p}{q}\ [\forall\text{–elim}]$$

Example 3.17 The universal quantifier distributes through conjunction. We will prove this in one direction only:

$$(\forall\, x : a \bullet p \wedge q) \Rightarrow ((\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q))$$

We begin the proof with the stated goal:

$$\frac{\vdots}{(\forall\, x : a \bullet p \wedge q) \Rightarrow ((\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q))}$$

The major connective here is the implication, so we assume the antecedent and try to prove the consequent:

$$\frac{\dfrac{\lceil \forall\, x : a \bullet p \wedge q \rceil^{[1]}}{\vdots}}{(\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q)}}{(\forall\, x : a \bullet p \wedge q) \Rightarrow ((\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q))}\ [\Rightarrow\text{–intro}^{[1]}]$$

In order to prove this conjunction, we should prove each conjunct separately:

$$\frac{\dfrac{\dfrac{\lceil \forall\, x : a \bullet p \wedge q \rceil^{[1]}}{\vdots}}{\forall\, x : a \bullet p} \qquad \dfrac{\dfrac{\lceil \forall\, x : a \bullet p \wedge q \rceil^{[1]}}{\vdots}}{\forall\, x : a \bullet q}}{(\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q)}\ [\wedge\text{–intro}]}{(\forall\, x : a \bullet p \wedge q) \Rightarrow ((\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q))}\ [\Rightarrow\text{–intro}^{[1]}]$$

We shall deal with the left-hand subtree. The major connective is the universal quantifier, so we introduce it.

$$\dfrac{\dfrac{\dfrac{\lceil \forall\, x : a \bullet p \wedge q \rceil^{[1]}}{\vdots}}{p}}{\dfrac{\forall\, x : a \bullet p}{(\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q)}\,[\wedge\text{-intro}]}\,[\forall\text{-intro}^{[2]}] \qquad \dfrac{\lceil \forall\, x : a \bullet p \wedge q \rceil^{[1]}}{\vdots}}{\forall\, x : a \bullet q}}{(\forall\, x : a \bullet p \wedge q) \Rightarrow ((\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q))}\,[\Rightarrow\text{-intro}^{[1]}]$$

We cannot work backwards any further, so now we must take advantage of our assumptions. If we eliminate the universal quantifier, we expose a useful conjunction:

$$\dfrac{\dfrac{\dfrac{\dfrac{\lceil x \in a \rceil^{[2]} \quad \lceil \forall\, x : a \bullet p \wedge q \rceil^{[1]}}{p \wedge q}\,[\forall\text{-elim}]}{\vdots}}{p}}{\dfrac{\forall\, x : a \bullet p}{(\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q)}\,[\wedge\text{-intro}]}\,[\forall\text{-intro}^{[2]}] \qquad \dfrac{\lceil \forall\, x : a \bullet p \wedge q \rceil^{[1]}}{\vdots}}{\forall\, x : a \bullet q}}{(\forall\, x : a \bullet p \wedge q) \Rightarrow ((\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q))}\,[\Rightarrow\text{-intro}^{[1]}]$$

Now this subtree is finished, since we can use conjunction elimination to connect top and bottom. The right-hand subtree is symmetric with the left.

$$\dfrac{\dfrac{\dfrac{\lceil x \in a \rceil^{[2]} \quad \lceil \forall\, x : a \bullet p \wedge q \rceil^{[1]}}{p \wedge q}\,[\forall\text{-elim}]}{p}\,[\wedge\text{-elim1}]}{\forall\, x : a \bullet p}\,[\forall\text{-intro}^{[2]}] \qquad \dfrac{\dfrac{\dfrac{\lceil x \in a \rceil^{[3]} \quad \lceil \forall\, x : a \bullet p \wedge q \rceil^{[1]}}{p \wedge q}\,[\forall\text{-elim}]}{q}\,[\wedge\text{-elim2}]}{\forall\, x : a \bullet q}\,[\forall\text{-intro}^{[3]}]}{(\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q)}\,[\wedge\text{-intro}]}{(\forall\, x : a \bullet p \wedge q) \Rightarrow ((\forall\, x : a \bullet p) \wedge (\forall\, x : a \bullet q))}\,[\Rightarrow\text{-intro}^{[1]}]$$

□

Example 3.18 Provided that x does not occur free in the antecedent, then we can move the universal quantifier through an implication

$$(\forall\, x : a \bullet p \Rightarrow q) \Leftrightarrow (p \Rightarrow \forall\, x : a \bullet q) \qquad \text{provided } x \text{ is not free in } p$$

A suitable proof would be:

$$\dfrac{\dfrac{\dfrac{

Chapter 4

Equality and Definite Description

In this chapter we extend our language of mathematics by adding a theory of equality between expressions. The language of predicate calculus with equality is strictly more expressive than without, since it allows us to assert the identity of two objects, or to distinguish between them. We provide inference rules to support the intuitive notion that expressions which are equal may be substituted one for the other, without affecting the truth of a statement, or the value of a larger expression. These rules form the basis of our theory of equality, and properties such as symmetry and transitivity can be derived from them.

The addition of equality allows us to formulate a simple rule for reasoning with quantifications: the one-point rule. We show how this rule may be used to introduce and eliminate the existential quantifier. We show also how equality may be used in statements expressing uniqueness and numerical quantity. We conclude the chapter by introducing a notation for identifying objects by using a description of their properties, rather than by referring to them by name.

4.1 Equality

The notion of equality is a familiar one: in arithmetic we learn that $1 + 1$ equals 2; in the Christian religion, the 25th of December equals Christmas Day. Such statements are meant to indicate that the two expressions concerned have the same value, or that they denote the same object. In a formal description, we identify expressions using the equality symbol:

$$1 + 1 = 2, \qquad \mathit{Christmas\ Day} = 25\mathit{th\ December}$$

We write $e = f$ when e is identical to f, in the sense that we cannot distinguish between them.

Example 4.1 In an identity parade, a witness may state that 'the man on the right is the man who stole my idea', making the following identification:

$$\text{the man on the right} \;=\; \text{the man who stole my idea}$$

That is, the man on the right is identical to the man who stole the idea. □

The '=' symbol is due to Robert Recorde, whose textbook on algebra The Whetstone of Witte, published in 1557, used the symbol for the first time. Recorde argued that a pair of parallel lines of the same length were suitable as the symbol for equality 'bicause noe 2 thynges can be moare equalle'.

We do not use equality to state that two predicates are identical: the propositional connective of equivalence is reserved for that purpose. Rather, we use equality to state that two values (such as numbers) are identical. Thus, we may write $5 + 3 = 3 + 5$, since both sides of the equation are expressions which denote values. These denoted values are the same, so the equality is true. Equalities form the atomic propositions in our logical language; the only other way of obtaining an atomic proposition is through set membership, described in Chapter 5.

Everything is identical to itself: thus, if t is any expression, then t is equal to t. This principle is known as the law of reflection:

$$\frac{}{t = t}\ [\text{eq-ref}]$$

It should be remarked that there are logics in which this principle does not hold. It is, however, an axiom of standard Z.

Example 4.2 In basic arithmetic, everybody knows that

$$1 + 1 \;=\; 1 + 1$$

whatever the properties of numbers and addition. □

Another axiom involving equality is Leibniz's law, or the substitution of equals: if $s = t$, then whatever is true of s is also true of t.

$$\frac{s = t \quad p[t \,/\, x]}{p[s \,/\, x]}\ [\text{eq-sub}]$$

In fact, Leibniz (1646–1716) stated the following Principle of the Identity of Indiscernibles: $s = t$ if and only if every property of s is a property of t, and conversely. The rule above follows from this principle.

Example 4.3 If we know that *Christmas Day* $=$ 25*th December*, and that

> 25*th December* falls on a Sunday this year

then we may apply the [eq-sub] rule and conclude that

> *Christmas Day* falls on a Sunday this year

□

If two expressions e and f are not identical, then we write $e \neq f$. This is simply an abbreviation for $\neg(e = f)$.

Example 4.4 Expressions with different properties are themselves different:

$$\frac{p[s \,/\, x] \quad \neg p[t \,/\, x]}{s \neq t}$$

This may be proved by

$$\dfrac{\dfrac{p[s \,/\, x] \qquad \dfrac{\dfrac{[s = t]^{[1]} \qquad \dfrac{\neg p[t \,/\, x]}{(\neg p)[t \,/\, x]}\,[\text{subst}]}{(\neg p)[s \,/\, x]}\,[\text{eq-sub}]}{\neg p[s \,/\, x]}\,[\text{subst}]}{\dfrac{\dfrac{\mathit{false}}{\neg(s = t)}\,[\neg\text{–intro}^{[1]}]}{s \neq t}\,[\text{abbreviation}]}\,[\neg\text{–elim}]$$

□

Using the rules [eq-ref] and [eq-sub], we are able to establish that equality is symmetric: for any expressions s and t, if $s = t$, then $t = s$. If we let x be a fresh variable, so that x does not appear in either s or t, then we may construct the following derivation:

$$\dfrac{s = t \qquad \dfrac{\dfrac{}{t = t}\,[\text{eq-ref}]}{(t = x)[t \,/\, x]}\,[\text{subst}]}{(t = x)[s \,/\, x]}\,[\text{eq-sub}]\quad\Bigg/\quad t = s\,[\text{subst}]$$

Having derived this property, we may use it as a rule of inference in our natural deduction system; we will refer to it as [eq-symm].

Example 4.5 From the identification made in Example 4.1, we may apply the [eq-symm] rule and conclude that:

the man who stole my idea = the man on the right

□

We are also able to establish that equality is transitive: for any expressions s, t and u, if $s = t$ and $t = u$ then $s = u$. Again, let x be a fresh variable:

$$\dfrac{s = t \qquad \dfrac{t = u}{(x = u)[t \,/\, x]}\,[\text{subst}]}{\dfrac{(x = u)[s \,/\, x]}{s = u}\,[\text{subst}]}\,[\text{eq-sub}]$$

Example 4.6 After the identity parade, it is revealed that the man on the right is Professor Plum, the prominent plagiarist:

the man on the right = Professor Plum

We may add this to the information in Example 4.5 and conclude that

the man who stole my idea = Professor Plum

□

4.3 Uniqueness and quantity

Equality can be used to make our predicate calculus more expressive, by allowing us to identify and distinguish objects.

Example 4.8 Let $x \mathit{\ loves\ } y$ mean that x is in love with y, and let *Person* be the set of all people. We may symbolise the proposition 'only Romeo loves Juliet' using a conjunction:

$$\begin{array}{l} \mathit{Romeo\ loves\ Juliet} \\ \wedge \\ \forall\, p : \mathit{Person} \bullet p \mathit{\ loves\ Juliet} \Rightarrow p = \mathit{Romeo} \end{array}$$

That is, any person who loves Juliet must be Romeo. □

A similar technique can be used to formalise statements containing the English phrases 'at most' and 'no more than'.

Example 4.9 The statement 'there is at most one person with whom Romeo is in love' may be formalised as

$$\forall\, p, q : \mathit{Person} \bullet \mathit{Romeo\ loves\ } p \wedge \mathit{Romeo\ loves\ } q \Rightarrow p = q$$

That is, if p and q are two people that Romeo loves, then they must be the same person. □

Example 4.10 The statement 'no more than two visitors are permitted' can be formalised as

$$\forall\, p, q, r : \mathit{Visitors} \bullet p = q \lor q = r \lor r = p$$

□

The notion of 'at least one' can be formalised using the existential quantifier.

Example 4.11 The statement 'at least one person has applied' could be formalised as

$$\exists\, p : \mathit{Person} \bullet p \in \mathit{Applicants}$$

□

However, to say 'at least two', we need equality.

Example 4.12 'There are at least two applicants' may be formalised as

$$\exists\, p, q : \mathit{Applicants} \bullet p \neq q$$

□

With the notions of 'at least' and 'at most', we have a method for formalising definite numerical propositions. This will be made easier later in the book when we have introduced finite sets. Then we shall be able to say, for example, that there are 29 distinct things with property p.

Example 4.13 The statement 'there is exactly one book on my desk' may be formalised as

$$\exists\, b : \mathit{Book} \bullet b \in \mathit{Desk} \land (\forall\, c : \mathit{Book} \mid c \in \mathit{Desk} \bullet c = b)$$

where 'Book' denotes the set of all books, and '$x \in \mathit{Desk}$' means that 'x is on my desk'. □

Specifying that there is exactly one object with a given property occurs so often that there is a special notation for it: the unique quantifier. We write

$$\exists_1\, x : a \bullet p$$

when there exists exactly one element x of set a such that p holds. This new quantifier can be defined in terms of the two forms of quantifier introduced in Chapter 3:

$$\exists_1 x : a \mid p \bullet q \;\Leftrightarrow\; \exists x : a \mid p \bullet q \wedge \forall y : a \bullet p[y / x] \wedge q[y / x] \Rightarrow y = x$$

The predicate $\exists_1 x : a \mid p \bullet q$ is true under two conditions: there must be an element x of a satisfying p and q and any element y of a that satisfies p and q is identical to x.